

```
import pandas as pd
import numpy as np

# 1. Load the CSV files into DataFrames
# This tells Pandas to read the files you just uploaded
df1 = pd.read_csv('German-Credit_1.csv')
df2 = pd.read_csv('German-Credit_2.csv')

# Let's print the shapes just to be sure they loaded correctly
print(f"Dataset 1 Shape: {df1.shape}")
print(f"Dataset 2 Shape: {df2.shape}")

# 2. Merge df1 and df2 using the shared 'OBS' column
# This combines all the demographic data with the banking history data!
merged_df = pd.merge(df1, df2, on='OBS')

# 3. Print the shape of the new merged dataset
# It should now be 1000 rows and 29 columns
print(f"\nShape of Merged Dataset: {merged_df.shape}")

# 4. View the first 5 rows of the fully combined dataset
display(merged_df.head())
```

Dataset 1 Shape: (1000, 23)
Dataset 2 Shape: (1000, 7)

Shape of Merged Dataset: (1000, 29)

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE	RADIO_TV	EDUCATION	RETRAINING	AMOUNT	INSTALL_RATE	...	NUM_DEPENDENTS	T
0	1	6.0	0.0	0.0	0.0	1.0	0.0	0.0	1169.0	4.0	...	1.0	
1	2	48.0	0.0	0.0	0.0	1.0	0.0	0.0	5951.0	2.0	...	1.0	
2	3	12.0	0.0	0.0	0.0	0.0	1.0	0.0	2096.0	2.0	...	2.0	
3	4	42.0	0.0	0.0	1.0	0.0	0.0	0.0	7882.0	2.0	...	2.0	
4	5	24.0	1.0	0.0	0.0	0.0	0.0	0.0	4870.0	3.0	...	2.0	

5 rows × 29 columns

--- Part (b) Data Exploration ---

```
# a) Find the average credit amount (AMOUNT)
avg_amount = merged_df['AMOUNT'].mean()
print(f"a) Average Credit Amount: {avg_amount:.2f}")

# b) Find the maximum and minimum AGE
max_age = merged_df['AGE'].max()
min_age = merged_df['AGE'].min()
print(f"b) Maximum Age: {max_age} | Minimum Age: {min_age}")

# c) Count customers with RESPONSE = 1
response_count = (merged_df['RESPONSE'] == 1).sum()
print(f"c) Customers with RESPONSE = 1: {response_count}")

# d) Find the distribution of INSTALL_RATE
print("\nd) Distribution of INSTALL_RATE:")
print(merged_df['INSTALL_RATE'].value_counts().sort_index())
```

a) Average Credit Amount: 3273.17
b) Maximum Age: 75.0 | Minimum Age: 19.0
c) Customers with RESPONSE = 1: 698

d) Distribution of INSTALL_RATE:
INSTALL_RATE
1.0 135
2.0 230
3.0 155
4.0 473
Name: count, dtype: int64

--- Part (c) Data Manipulation ---

```
# a) Create a new column: AMOUNT_PER_DURATION
# We simply divide the AMOUNT column by the DURATION column
merged_df['AMOUNT_PER_DURATION'] = merged_df['AMOUNT'] / merged_df['DURATION']
print("\nc.a) Added 'AMOUNT_PER_DURATION'. Here are the first 3 rows showing the new column:")
display(merged_df[['AMOUNT', 'DURATION', 'AMOUNT_PER_DURATION']].head(3))

# b) Filter customers with AGE > 40 AND AMOUNT > 5000
```

```
# Remember to wrap each condition in parentheses and use '&'
filtered_customers = merged_df[(merged_df['AGE'] > 40) & (merged_df['AMOUNT'] > 5000)]
print(f"\nc.b) Number of customers older than 40 with > 5000 credit: {filtered_customers.shape[0]}")
```

```
# c) Sort data based on AMOUNT (descending)
# ascending=False forces it to go from highest to lowest
sorted_amount_df = merged_df.sort_values(by='AMOUNT', ascending=False)
print("\nc.c) Top 3 highest credit amounts:")
display(sorted_amount_df[['OBS', 'AMOUNT']].head(3))
```

```
# --- Part (d) Grouping & Analysis ---
```

```
# a) Group by RESPONSE and find mean AMOUNT
mean_amount_by_response = merged_df.groupby('RESPONSE')['AMOUNT'].mean()
print("\nd.a) Mean AMOUNT grouped by RESPONSE:")
print(mean_amount_by_response)
```

```
# b) Find average AGE for each RESPONSE
mean_age_by_response = merged_df.groupby('RESPONSE')['AGE'].mean()
print("\nd.b) Average AGE grouped by RESPONSE:")
print(mean_age_by_response)
```

c.a) Added 'AMOUNT_PER_DURATION'. Here are the first 3 rows showing the new column:

	AMOUNT	DURATION	AMOUNT_PER_DURATION
0	1169.0	6.0	194.833333
1	5951.0	48.0	123.979167
2	2096.0	12.0	174.666667

c.b) Number of customers older than 40 with > 5000 credit: 60

c.c) Top 3 highest credit amounts:

	OBS	AMOUNT
915	916	18424.0
95	96	15945.0
818	819	15857.0

```
d.a) Mean AMOUNT grouped by RESPONSE:
RESPONSE
0.0    3946.831081
1.0    2986.940837
Name: AMOUNT, dtype: float64
```

```
d.b) Average AGE grouped by RESPONSE:
RESPONSE
0.0    33.787162
1.0    36.207792
Name: AGE, dtype: float64
```

```
# --- Part (e) Merging & Advanced ---
```

```
# e.b) Find correlation between numerical variables
# We use .select_dtypes() to ensure we don't try to correlate text/categorical data
numeric_df = merged_df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()

print("e.b) Correlation Matrix (Preview of first 5x5):")
display(correlation_matrix.iloc[:5, :5])
```

```
# --- Part (f) NumPy Tasks ---
```

```
print("\n" + "*" * 40 + "\n--- Part (f) NumPy Tasks ---\n" + "*" * 40)
```

```
# f.a) Create a 3x4 NumPy array and print shape, size, dtype
arr_3x4 = np.array([[1, 2, 3, 4],
                    [5, 6, 7, 8],
                    [9, 10, 11, 12]])
print("f.a) 3x4 Array:\n", arr_3x4)
print(f"Shape: {arr_3x4.shape} | Size: {arr_3x4.size} | Dtype: {arr_3x4.dtype}")
```

```
# f.b) Create arrays using zeros(), ones(), and full()
zeros_arr = np.zeros((2, 2))
ones_arr = np.ones((2, 3))
full_arr = np.full((2, 2), 7) # Fills a 2x2 array with the number 7
print("\nf.b) Zeros Array:\n", zeros_arr)
print("Ones Array:\n", ones_arr)
print("Full Array (filled with 7s):\n", full_arr)
```

```
# f.c) Generate numbers using arange() and linspace()
arange_arr = np.arange(0, 10, 2) # Start 0, Stop 10 (exclusive), Step 2
linspace_arr = np.linspace(0, 1, 5) # 5 evenly spaced numbers between 0 and 1
print("\nf.c) Arange Array (0 to 10, step 2):", arange_arr)
print("Linspace Array (0 to 1, 5 values):", linspace_arr)

# f.d) Create a random array and fix randomness using seed
np.random.seed(42) # This locks the random numbers so they are the same every time
random_arr = np.random.randint(0, 10, size=(3, 3))
print("\nf.d) Random Array (Seed=42):\n", random_arr)

# f.e) Perform indexing (first, last, and specific element)
first_elem = arr_3x4[0, 0]
last_elem = arr_3x4[-1, -1]
specific_elem = arr_3x4[1, 2] # Row index 1 (2nd row), Column index 2 (3rd column)
print(f"\nf.e) First element: {first_elem} | Last element: {last_elem} | Element at [1,2]: {specific_elem}")

# f.f) Slice a 2D array to extract a submatrix
submatrix = arr_3x4[0:2, 1:3] # Rows 0 and 1, Columns 1 and 2
print("\nf.f) Submatrix (Rows 0-1, Cols 1-2):\n", submatrix)

# f.g) Reshape a 1D array into 2D array
arr_1d = np.arange(1, 13) # 1D array of 12 numbers (1 through 12)
reshaped_2d = arr_1d.reshape((3, 4))
print("\nf.g) Reshaped 1D to 3x4 2D Array:\n", reshaped_2d)

# f.h) Perform transpose of an array
transposed_arr = arr_3x4.T
print("\nf.h) Transposed Array (Columns become rows):\n", transposed_arr)
```

e.b) Correlation Matrix (Preview of first 5x5):

	OBS	DURATION	NEW_CAR	USED_CAR	FURNITURE
OBS	1.000000	0.030324	0.058738	0.008841	-0.007529
DURATION	0.030324	1.000000	-0.111228	0.145838	-0.059562
NEW_CAR	0.058738	-0.111228	1.000000	-0.187518	-0.259281
USED_CAR	0.008841	0.145838	-0.187518	1.000000	-0.158949
FURNITURE	-0.007529	-0.059562	-0.259281	-0.158949	1.000000

```
=====
--- Part (f) NumPy Tasks ---
=====
```

f.a) 3x4 Array:
[[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]
Shape: (3, 4) | Size: 12 | Dtype: int64

f.b) Zeros Array:
[[0. 0.]
[0. 0.]]
Ones Array:
[[1. 1. 1.]
[1. 1. 1.]]
Full Array (filled with 7s):
[[7 7]
[7 7]]

f.c) Arange Array (0 to 10, step 2): [0 2 4 6 8]
Linspace Array (0 to 1, 5 values): [0. 0.25 0.5 0.75 1.]

f.d) Random Array (Seed=42):
[[6 3 7]
[4 6 9]
[2 6 7]]

f.e) First element: 1 | Last element: 12 | Element at [1,2]: 7

f.f) Submatrix (Rows 0-1, Cols 1-2):
[[2 3]
[6 7]]

f.g) Reshaped 1D to 3x4 2D Array:
[[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]

f.h) Transposed Array (Columns become rows):
[[1 5 9]
[2 6 10]
[3 7 11]
[4 8 12]]

